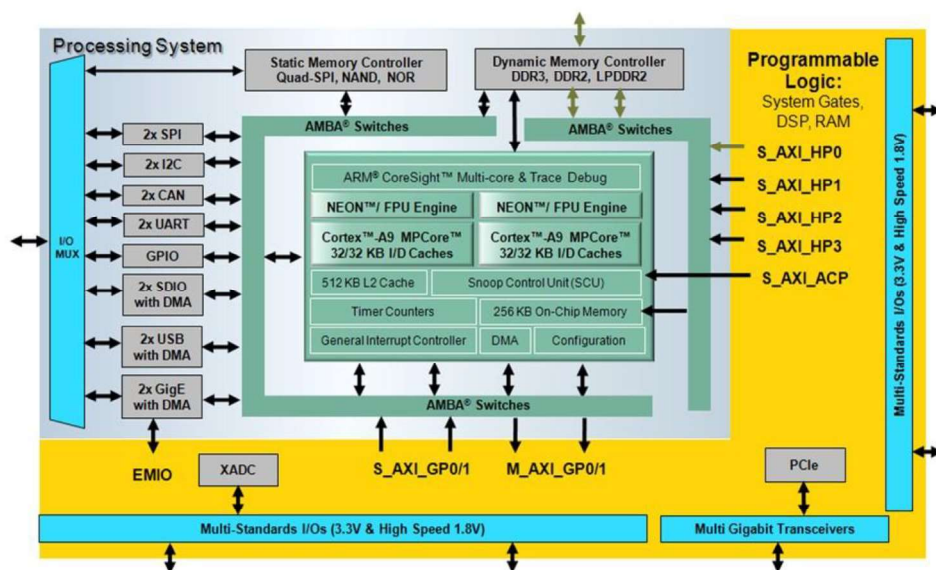


Zynq SoC Overview

2021.2

Zynq-7000 SoC Block Diagram



Zynq SoC PS and PL

> The Zynq® SoC combines a state-of-the-art processor system (PS) and programmable logic (PL) into a single cohesive device

> PS – Processor system

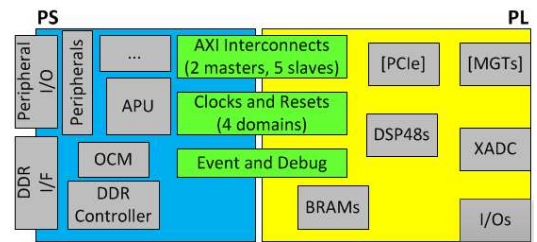
- Dual ARM Cortex-A9 processors plus caches, on-chip memory, and other peripherals
- Hard silicon
- Masters the device (boots first, controls loading of PL)

> PL – Programmable logic

- Shares the same 7 series programmable logic as Artix® and Kintex®-based devices

> PS and PL represent two domains that define

- A division of both architecture and physical die geometry
- Separate power domains

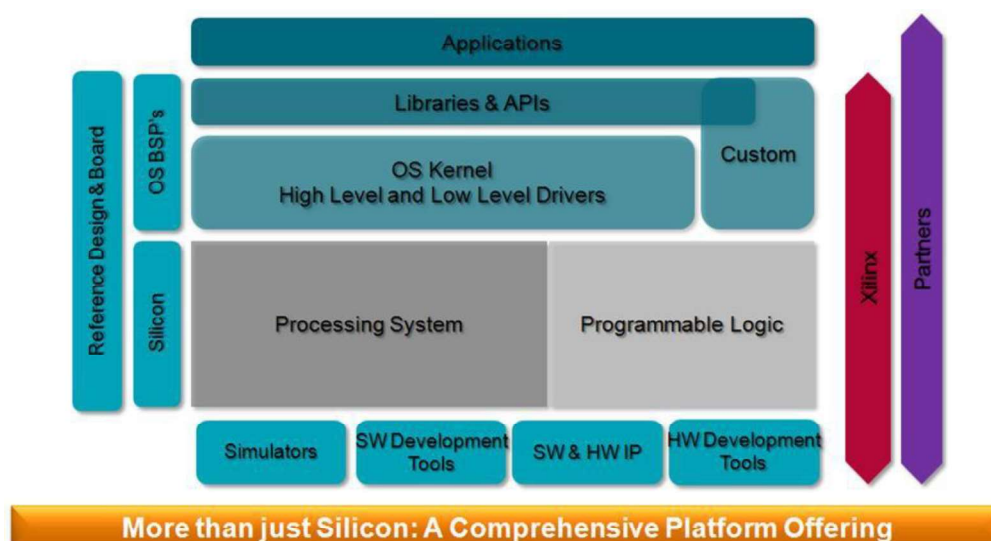


>> Zynq SoC Overview - 1-3

© Copyright 2022 Xilinx

133282
XILINX

Zynq-7000 SoC Offering



>> Zynq SoC Overview - 1-4

© Copyright 2022 Xilinx

133282
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

PS Components

- > The PS can be roughly divided into four major functional units
 - Application processor unit (APU)
 - I/O peripherals (IOP)
 - Datapath (interconnects and interfaces)
 - Memory resources
- > Other more minor units include clock generation, reset system, and device configuration
- > The APU is the core of the PS and houses the dual Arm® Cortex™-A9 MPCore processor
- > The choice of Arm processor technology provides a large and active ecosystem
- > The PS components remain the same for all devices in the Zynq SoC family
 - Only the speed grade of the PS may vary

Zynq-7000 Family Highlights

- > Complete Arm®-based processing system
 - Application processor unit
 - Dual Arm Cortex™-A9 processors
 - Caches and support blocks
 - Fully integrated memory controllers
 - I/O peripherals
- > Tightly integrated programmable logic
 - Used to extend the processing system
 - Scalable density and performance
- > Flexible array of I/O
 - Wide range of external multi-standard I/O
 - High-performance integrated serial transceivers
 - Analog-to-digital converter inputs

Zynq-7000 SoC Product Table – PS Capabilities

Processing System (PS)

Features	Zynq-7000S	Zynq-7000	
Devices	Z-7007S, Z-7012S, Z-7014S	Z-7010, Z-7015, Z-7020	Z-7030, Z-7035, Z-7045, Z-7100
Processor Core	Single-core ARM® Cortex™-A9 MPCore™	Dual-core ARM Cortex-A9 MPCore	
Maximum Frequency	Up to 766MHz	Up to 866 MHz	Up to 1GHz
External Memory Support	DDR3, DDR3L, DDR2, LPDDR2		
Key Peripherals	USB 2.0, Gigabit Ethernet, SD/SDIO		
Dedicated Peripheral Pins	Up to 128	Up to 128	128

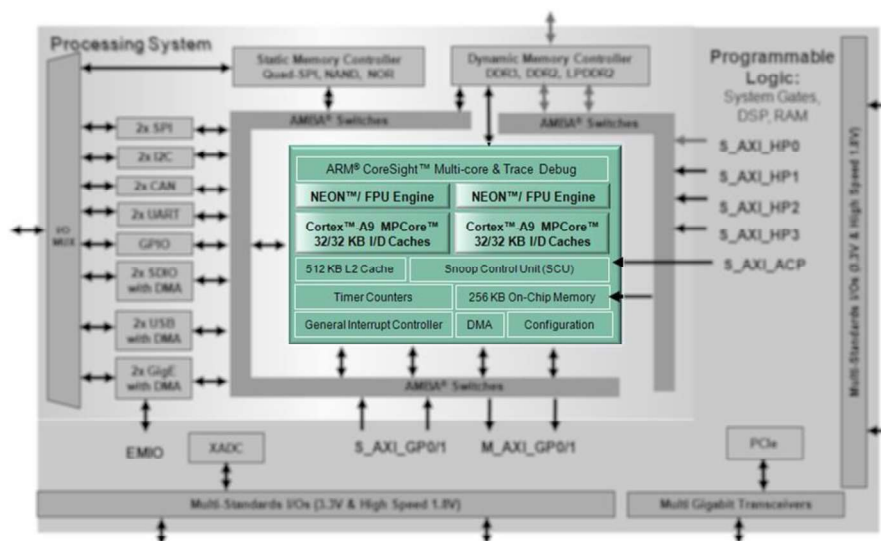
>> Zynq SoC Overview - 1-7

© Copyright 2022 Xilinx

133282



Zynq SoC Block Diagram – APU



>> Zynq SoC Overview - 1-8

© Copyright 2022 Xilinx

133282



DO NOT DISTRIBUTE PERSONAL USE ONLY

PS APU Components

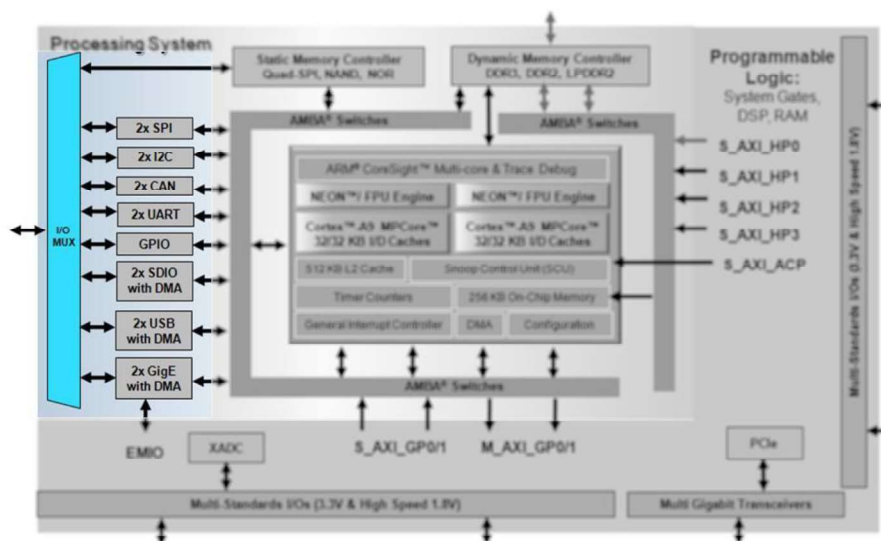
- > The APU is the most critical component of the PS and contains several major components including
 - Dual Arm Cortex-A9 central processing units (CPUs)
 - Cache and cache controllers
 - General interrupt controller (GIC)
 - Timers (public and private)
- > Each processor core includes
 - Single/double precision vector floating point (VFP) extension
 - Advanced Single Instruction Multiple Data (SIMD) extension
 - Instruction and data cache
- > Other components often, but not always, considered part of the APU include
 - On-chip memory (OCM)
 - Direct memory access (DMA) unit

>> Zynq SoC Overview - 1-9

© Copyright 2022 Xilinx

133282
XILINX

Zynq SoC Block Diagram – IOP



>> Zynq SoC Overview - 1-10

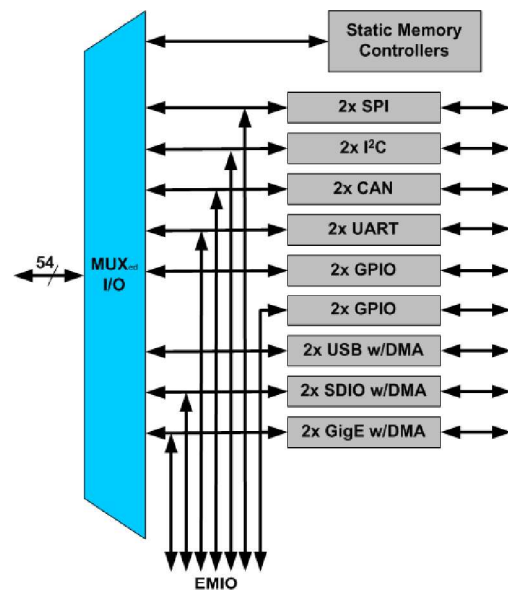
© Copyright 2022 Xilinx

133282
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Zynq SoC Built-in Peripherals (1)

- > Two USB 2.0 OTG / device / host
- > Two tri-mode gigabit Ethernet (10/100/1000)
- > Two SD/SDIO interfaces
 - Memory, I/O, and combo cards
- > Two CAN 2.0Bs, SPIs, I2Cs, UARTs
- > Four GPIO 32-bit blocks
 - 54 available through MIO; other 64 available through EMIO



>> Zynq SoC Overview - 1-11

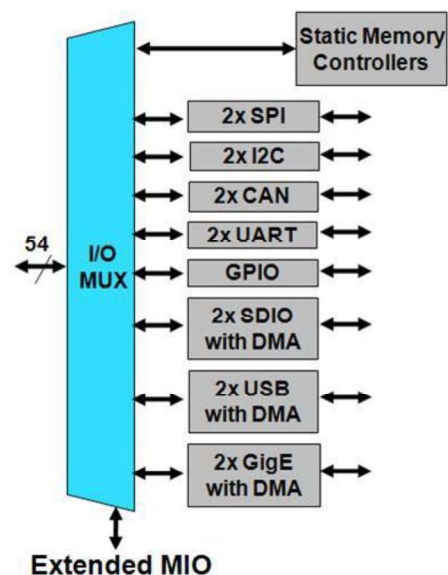
© Copyright 2022 Xilinx

133282

XILINX

Zynq SoC Built-in Peripherals (2)

- > Multiplexed input/output (MIO)
 - Multiplexed output of peripheral and static memories
 - Two I/O banks; each selectable: 1.8V, 2.5V, or 3.3V
 - Configured using new feature in the Vivado® IP integrator
- > Extended MIO
 - Enables use of the SelectIO™ interface with PS peripherals



>> Zynq SoC Overview - 1-12

© Copyright 2022 Xilinx

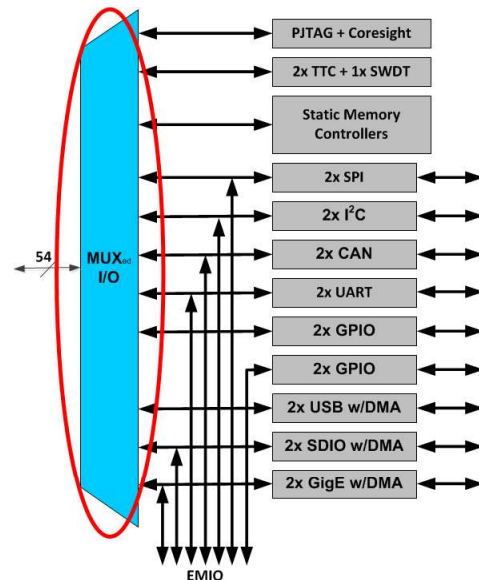
133282

XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Multiplexed I/O (MIO)

- > External interface to PS I/O peripheral ports
 - 54 dedicated package pins available
 - Not a fully populated MUX (not all connections possible)
 - Software configurable
 - Default configuration assigned in the Vivado Design Suite passed via
 - Tcl script when debugging
 - C object code for delivery
 - Not available for all peripheral ports
 - Some ports can only use EMIO
 - Some peripherals have additional capabilities through EMIO



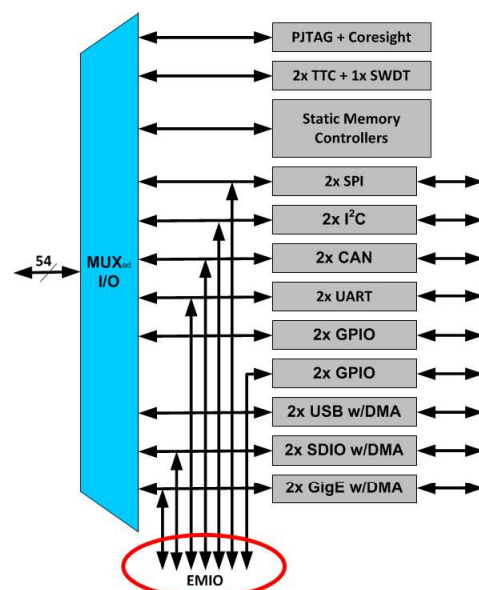
>> Zynq SoC Overview - 1-13

© Copyright 2022 Xilinx

133282
XILINX

Extended Multiplexed I/O (EMIO)

- > Extended interface to PS I/O peripheral ports
 - EMIO: Peripheral port to programmable logic
 - Alternative to using MIO
 - Mandatory for some peripheral ports
 - Facilitates
 - Connection to peripheral in programmable logic
 - Use of general I/O pins to supplement MIO pin usage
 - Allows additional signals for many of the peripherals
 - Alleviates competition for MIO pin usage



>> Zynq SoC Overview - 1-14

© Copyright 2022 Xilinx

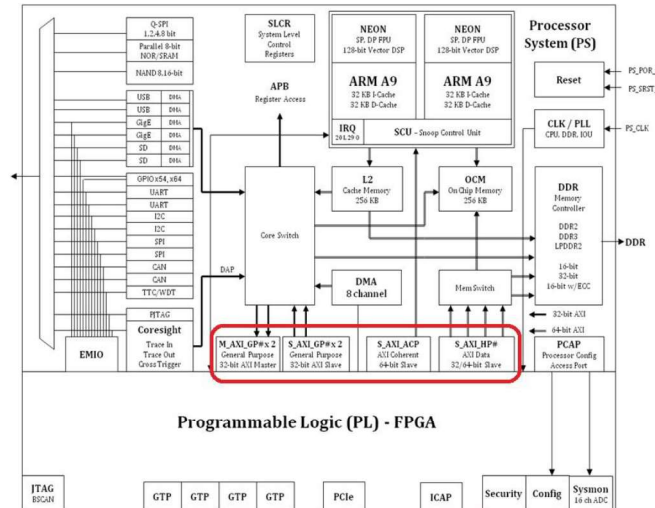
133282
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

DO NOT DISTRIBUTE PERSONAL USE ONLY

AXI Ports of the PS-PL Interface

- > The AMBA AXI ports of the PS-PL interface provide the primary mechanism for the flow of data between the PS and PL



>> Zynq SoC Overview - 1-17

© Copyright 2022 Xilinx

133282
XILINX

PS Memory Resources

- > Memory resources in the PS consist of memory interfaces and embedded memory
- > Two memory interfaces for external memory
 - DDRx dynamic memory controller
 - Supports LPDDR2, DDR2, DDR3
 - Flash / static, memory controller
 - Supports SRAM, QSPI, NAND/NOR FLASH
- > Two embedded memory components
 - On-chip memory (OCM)
 - 256KB SRAM
 - ROM for boot code (BootROM)
 - Worth noting that the PS can utilize the PL to expand embedded memory resources (block RAM)

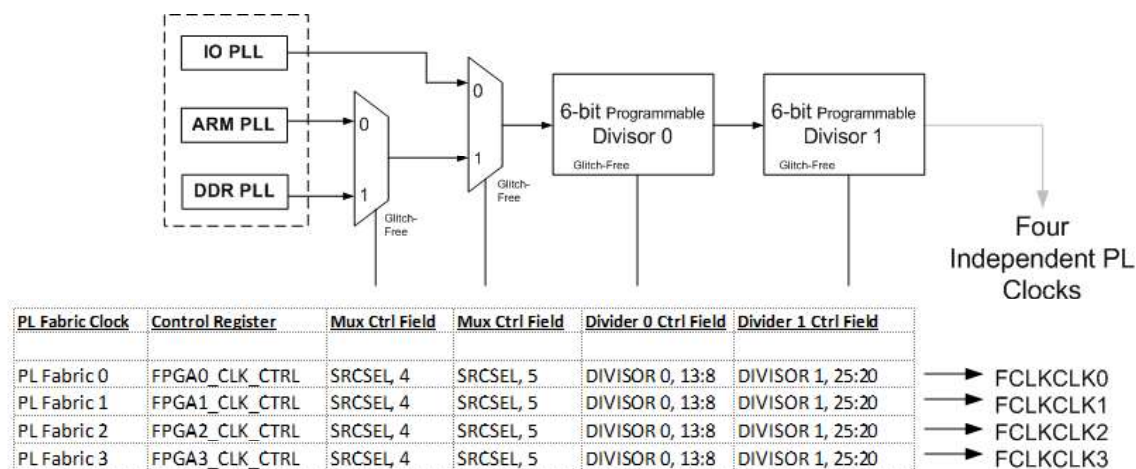
>> Zynq SoC Overview - 1-18

© Copyright 2022 Xilinx

133282
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Clocking the PL



PL Clocking Sources

- > PS clocks
 - PS clock source from external package pin
 - PS has three PLLs for clock generation
 - PS has four clock programmable ports to PL
 - Each clock is equipped with its own reset output
- > The PL has 7 series clocking resources
 - PL has a different clock source domain compared to the PS
 - The clock to PL can be sourced from external clock capable pins
 - Can use one of the four PS clocks as source
- > Synchronizing the clock between PL and PS is taken care of by the architecture of the PS
- > PL cannot supply clock source to PS

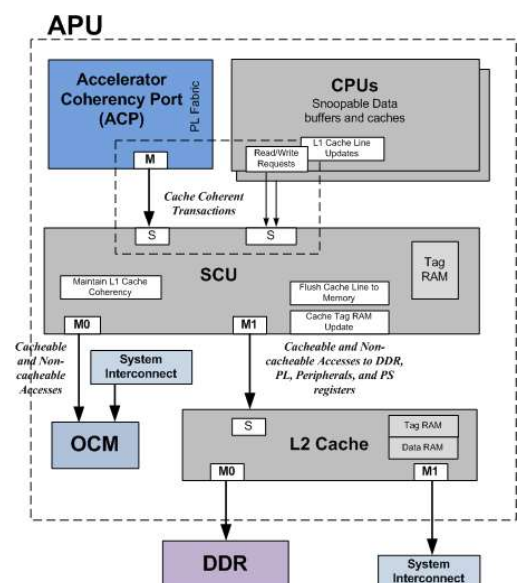
Zynq SoC Application Processor Unit (APU)

2021.2



APU – More Than a Couple of Processors

- > Heart of the processing system
- > Tightly coupled processors and sub-components for maximum performance
 - L1 cache for each processor
 - L2 cache shared by processors
 - Snooper control unit regulates memory flow, providing cache coherency
- > Tied to other PS components and PL via the PS interconnect



The Application Processor Unit (APU)

- > Dual Arm Cortex-A9 MPCore with NEON extensions
 - Up to 1 GHz operation
 - 2.5 DMIPS / MHz per core
 - Separate 32 KB instruction and data caches
- > Snoop control unit
 - L1 cache snoop control
 - Accelerator coherency port
- > Level 2 cache and controller
 - Shared 512 KB cache with parity



APU Sub-Components

- > General interrupt controller (GIC)
- > On-chip memory (OCM): RAM and boot ROM
- > Central DMA (eight channels)
- > Device configuration (DEVCFG)
- > Private watchdog timer and timer for each CPU
- > System watchdog, global timer, and triple timer counters shared between CPUs
- > Arm CoreSight™ debug technology

Arm Cortex-A9 Processor Features

- > Dual-core processor cluster
- > 2.5 DMIPS / MHz per processor
- > Modified-Harvard architecture
- > Self-contained L1 caches for instructions (32K) and data (32K)
- > 512 KB L2 shared cache
 - shared between processors
 - used for both instruction and data caching
- > Automatic cache coherency between processor cores
- > 1 GHz operation (fastest speed grade)

>> Zynq SoC Application Processor Unit (APU) - 1-5

© Copyright 2022 Xilinx

133443
 XILINX

Cortex-A9 Processor Companion Entities

- > Advanced L2 cache controller
 - The PL310 L2 controller was developed for use with the Cortex-A9 processor to ensure high data throughput and maximum performance
 - Used for both instruction and data
 - 512 KB shared among all processors in the cluster
- > Cortex-A9 NEON processing engine (NPE) contains
 - Vector Processor (integer and single precision floating point)
 - Floating point processor (double precision)
- > Arm CoreSight technology debug engine
 - Program trace macrocell (PTM)
 - Arm debug access port (DAP)
 - AMBA trace bus (ATB) AMBA3 protocol
 - Instrumentation trace macrocell
 - Core switch trace

>> Zynq SoC Application Processor Unit (APU) - 1-6

© Copyright 2022 Xilinx

133443
 XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Memory Management Unit

- > Use of MMU is optional but required by many operating systems
 - Provides and protects individual application (virtual) memory spaces
 - Maps virtual memory addresses to physical memory addresses
- > Address translations are done on a page-by-page basis
 - Translation tables stored in main memory maintain page mappings
 - Support for 16 KB, 64 KB, 1 MB, and 16 MB pages
 - Page tables must be initialized/populated prior to enabling MMU
- > Translation look-aside buffers (TLBs) cache recent mappings
 - Main TLB has 128 entries
 - Micro TLBs for instruction and data accesses have 32 entries
 - TLBs must be invalidated before use (done once by boot ROM)
- > Using the MMU increases performance
 - Enables use of data cache (recommended for most applications)
 - Allows optimization of memory accesses (e.g. out-of-order access)

>> Zynq SoC Application Processor Unit (APU) - 1-7

© Copyright 2022 Xilinx

133443
 XILINX

TrustZone

- > TrustZone is Arm technology integrated into the Cortex-A9 processor for increased system-wide security protection
- > Allows two states of operation for the processor
 - Secure World: Access to the entire address space
 - Normal World: Access only to non-secure addresses
- > AMBA AXI interface includes an additional secure / non-secure control bit that allows IP cores to support TrustZone
 - Supported by Xilinx LogiCore IP AXI Interconnect
 - Peripherals in PL can indirectly support TrustZone through the AXI Interconnect
- > DDR can be sub-divided into 64 MB secure / non-secure regions

>> Zynq SoC Application Processor Unit (APU) - 1-8

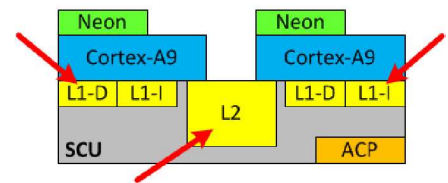
© Copyright 2022 Xilinx

133443
 XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Processor Caches

- > L1 cache
 - Smallest size but fastest access
 - 32 KB each for instruction and data
 - One I-cache and D-cache per CPU
- > L2 cache
 - Secondary, larger cache
 - 512 KB
 - Used for both instruction and data
 - One per snoop control unit (SCU)



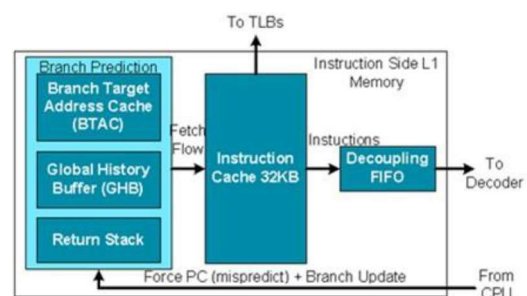
>> Zynq SoC Application Processor Unit (APU) - 1-9

© Copyright 2022 Xilinx

133443
XILINX

L1 Cache Features

- > Separate instruction and data caches for each processor
- > Each cache can be disabled independently, using the system control coprocessor



- > Caches are four-way, set associative, and are write back
- > L1 caches are not lockable
- > Cache line lengths for both L1 caches are 32 bytes
- > On a cache miss, critical word first filling of the cache is performed followed by the next word in the sequence
- > Both L1 caches support parity

>> Zynq SoC Application Processor Unit (APU) - 1-10

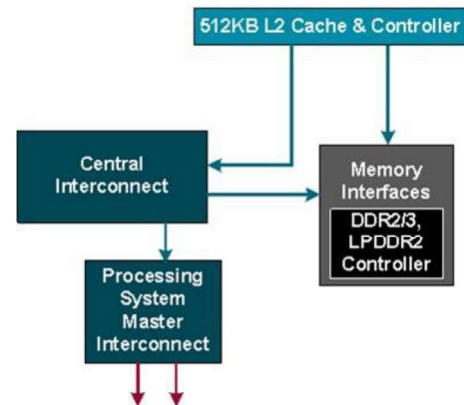
© Copyright 2022 Xilinx

133443
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

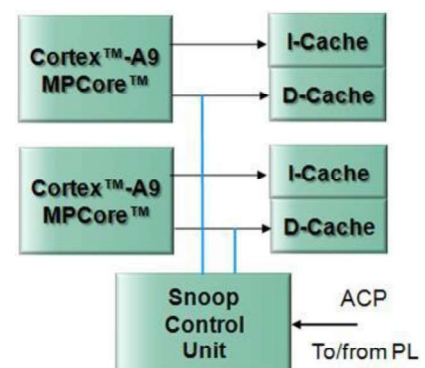
L2 Cache Features

- > L2 cache controller
 - 512 KB of RAM built into the SCU
 - Latency of 25 CPU cycles
 - Caches both instruction and data accesses
 - Fixed, 256-bit cache line size
 - Support for per-master locking by way or line
 - Eight-way, set associative
- > Two AXI interfaces
 - One AXI interface to the DDR controller
 - One AXI interface is to programmable logic master (to peripherals)



Cache Coherency Between Processors

- > Snoop control unit features
 - L1 cache snoop control
 - Coherent option for multi-core software development
 - Snoop filtering monitors cache traffic
 - High-performance, cache-to-cache transfers
 - Extra hardware to snoop each CPU and to cache each interface independently
 - If a cache line is updated, the SCU will know
 - The SCU will then take the updated cache line and update in the other CPU's cache



Controlling the PS Behavior

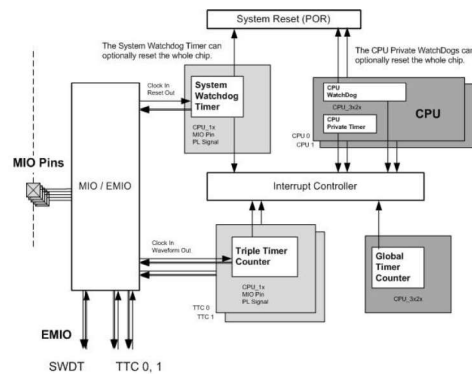
- > System-level control registers (SLCR)
 - A set of special registers in the APU used to configure the PS
 - Part of the systems addressable memory space
- > Examples of what can be configured through SLCRs
 - Power and clock management
 - Reset control
 - Multiplexed I/O (MIO) management
- > Accessible through software
 - Initial values of many SLCRs can be configured with the Vivado IP integrator
 - Low-level software provided by a Board Support Package (BSP) handles run-time changes

SLCR Register Categories

- > The SLCR block contains a number of registers that can be grouped into the following categories
 - System clock and reset control/status registers
 - TrustZone control register
 - APU control registers
 - SoC debug control registers
 - DMA initialization registers
 - MIO/IOP control / status registers
 - DDR control registers
 - Miscellaneous control registers
 - PL reset registers
 - RAM and ROM control registers

APU Timers

- > Global timer
- > Two triple time counters (TTCs)
- > SWDT: system watchdog timer
- > Per Cortex-A9 processor
 - AWDT: APU watchdog timer
 - Private timer



>> Zynq SoC Application Processor Unit (APU) - 1-15

© Copyright 2022 Xilinx

133443
XILINX

Global Timers

- > Shared by both processors
- > 64-bit incrementing counter
- > Independent, 64-bit compare register; issues interrupts for each processor
- > Great idea for real-time, heartbeat interrupt

>> Zynq SoC Application Processor Unit (APU) - 1-16

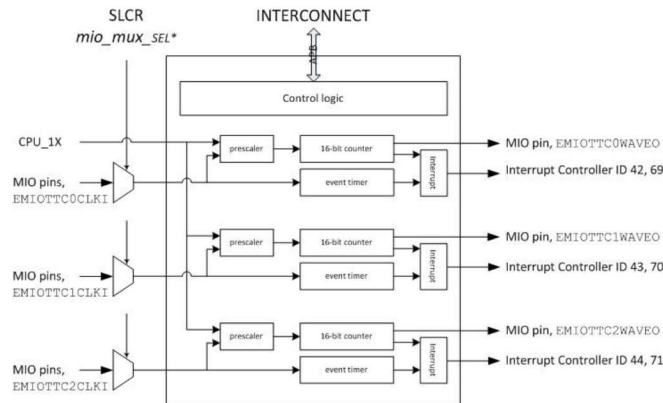
© Copyright 2022 Xilinx

133443
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Triple Time Counter (TTC)

- > The TTC has the following features
 - Both incrementing and decrementing count
 - Generates PWM output through the MIO
 - External or PL (MIO/EMIO) reference clock inputs and waveform outputs



>> Zynq SoC Application Processor Unit (APU) - 1-17

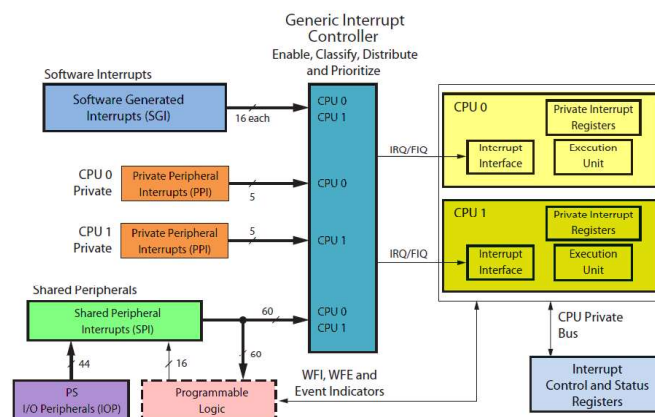
© Copyright 2022 Xilinx

133443

XILINX

Generic Interrupt Controller (GIC) (1)

- > Developed by Arm for Cortex-A/R
 - Integrated into the PS of Zynq® devices
- > Global and private configuration spaces
 - Global settings apply to all processor cores
 - Private CPU interface is per core
- > The GIC has four primary responsibilities over interrupt sources
 - Enable, classify, prioritize, and distribute



>> Zynq SoC Application Processor Unit (APU) - 1-18

© Copyright 2022 Xilinx

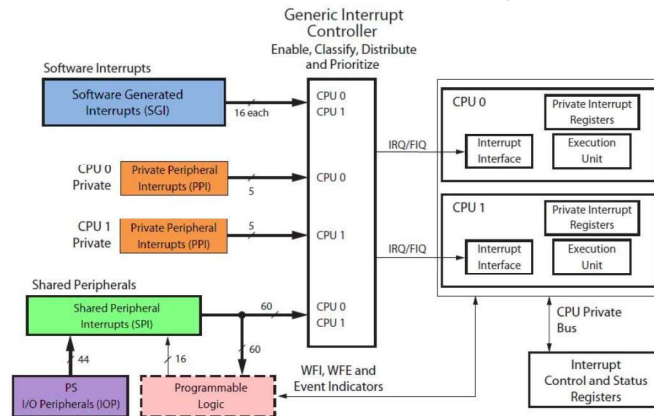
133443

XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Generic Interrupt Controller (GIC) (2)

- > Handles up to 16 software-generated interrupts (SGI)
- > Supports 60 shared peripheral interrupts (SPI) starting at ID 32
- > Processes both level-sensitive interrupts and edge-sensitive interrupts
- > Five private peripheral interrupts (PPI) dedicated for each core (no user-selectable PPI)



>> Zynq SoC Application Processor Unit (APU) - 1-19

© Copyright 2022 Xilinx

133443

XILINX

XILINX

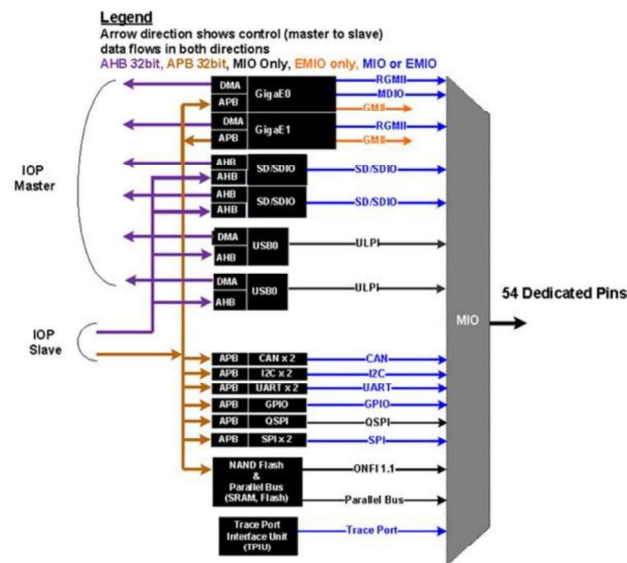
Zynq SoC Processor Input/Output Peripherals

2021.2

DO NOT DISTRIBUTE PERSONAL USE ONLY

Input/Output Peripherals

- > Two GigE
- > Two USB
- > Two SPI
- > Two SD/SDIO
- > Two CAN
- > Two I2C



- > Two UART
- > Four 32-bit GPIOs
- > Static memories
 - NAND
 - NOR/ SRAM
 - Quad SPI
- > Trace ports

>> Zynq SoC Processor Input/Output Peripherals - 1-2

© Copyright 2022 Xilinx

133444



Connecting to IOP (1)

- > IOP ports are accessed via the multiplexed I/O (MIO) or extensible multiplexed I/O (EMIO)
 - While most ports can use either, some are restricted to one or the other
 - For more details, see the *Zynq Technical Reference Manual* (UG585)
- > There are 54 MIO ports
- > MIO ports are available before the EMIO connection in programmable logic
- > MIO pin usage is **NOT** a fully populated crossbar
 - MIO selections vary by peripheral
 - Contention for the same MIO pins is common
 - Solve by using EMIO

>> Zynq SoC Processor Input/Output Peripherals - 1-3

© Copyright 2022 Xilinx

133444



DO NOT DISTRIBUTE PERSONAL USE ONLY

Connecting to IOP (2)

- > Many IOP have optional control ports
 - Consider which are needed for signal interface or PHY
- > EMIO port can access PL package pins or the user design
 - Not functional until the PL is configured

Multiplexed I/O (MIO) Pinout

IP	MIO	Extendable MIO in Programmable Logic
QSPI NOR/SRAM NAND	Yes	No
USB: 0,1	Yes, Phy off chip	No
SDIO: 0,1	Yes – 50MHz	Yes – 25MHz
SPI: 0,1 I2C: 0,1 CAN: 0,1 GPIO	Yes	Yes
GigE: 0,1	RGMI v2.0 (HSTL) Phy off chip	Supports GMII, RGMI v2.0 (HSTL), RGMI v1.3 (LVCMOS), RMII, MII, SGMI with wrapper in Programmable Logic
UART: 0,1	Simple UART: Only 2 pins (Tx & Rx)	Full UART (Tx, Rx, DTR, DCD, DSR, RI, RTS & CTS) either require: • 2 Processing System pins (Rx & Tx) through MIO + 6 additional Programmable Logic pins • 8 Programmable Logic pins

Do Not Forget the PHY (Physical Interface)

- > Many of the IOP block's peripherals require physical interface circuitry external to the Zynq SoC processing platform
- > A PHY could be any of the following
 - Nothing: Electrical standard of the Zynq architecture supports internal PHY
 - External memory components
 - Simple port translation (buffering) to an electrical standard
 - More complex port translation (buffering) to a protocol standard
 - Interface to an industry-standard PHY
- > MIO pins support 1.8V HSTL and LVCMOS standards, as well as 2.5V/3.3V standards
- > In addition to the MIO pins, most IOP interface signals are available to the PL, allowing use of PL-selected I/O pins

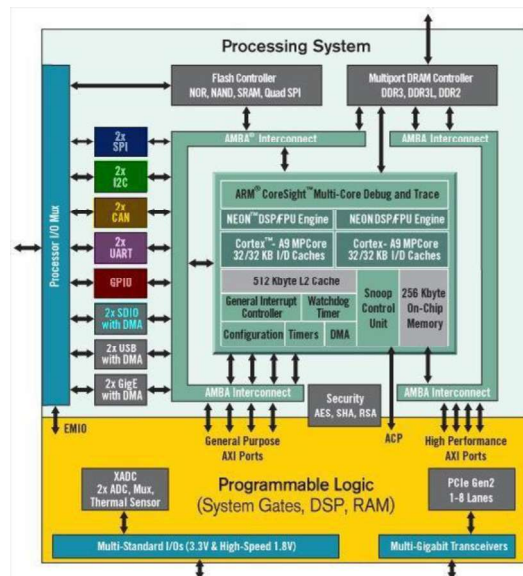


Low-Speed Peripherals: Overview

2021.2

Low-Speed Peripherals on the Zynq SoC (6)

- > Two full-duplex SPI controllers
 - Four chip selects
- > Two I2C controllers
 - Supports both master and slave
- > Two CAN 2.0B-compliant controllers
 - ISO 118981-1 standard
 - Supports an external PHY interface
- > Two UART controllers
- > Two GPIO
 - 54 signals to the PS pins
 - 64 signals to the PL
 - Shared with peripherals*
- > Two SD/SDIO 2.0-compliant controllers



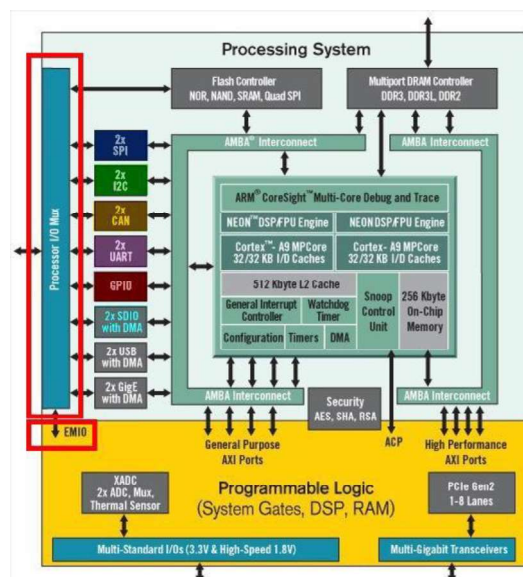
>> Low-Speed Peripherals: Overview - 1-2

© Copyright 2022 Xilinx

124026
XILINX

Low-Speed Peripherals on the Zynq SoC (7)

- > 54 GPIO signals for device pins (routed through MIO)
 - Outputs are 3-state capable
- > 192 GPIO signal between the PS and PL via the EMIO interface
 - 64 inputs, 128 outputs



>> Low-Speed Peripherals: Overview - 1-3

© Copyright 2022 Xilinx

124026
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Hardware IP Device Drivers

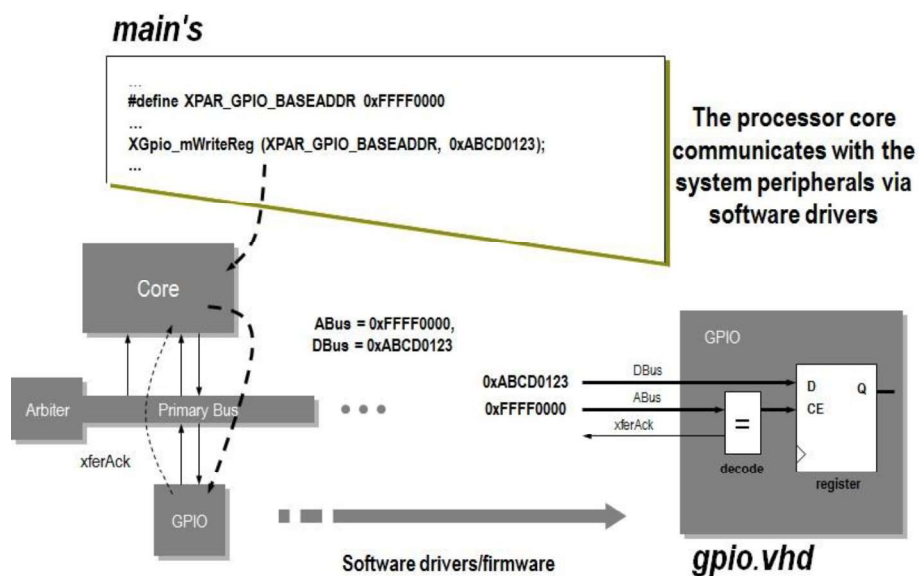
> Driver

- Provides an interface for the software to communicate with the hardware
- Designed to be portable across processor architectures and operating systems

> Delivery format

- Delivered as source code, allowing it to be built and optimized
 - Minimized assembly language
 - C programming language

Software Drivers



Kernel Details (Device Drivers)

Hardware Block	Driver Name
GPIO	<i>gpio-xilinuxps.c</i>
UART	<i>xilinx_uartps.c</i>
CAN	<i>xilinx_can.c</i>
I ² C	<i>i2c-cadence.c</i>
SPI	<i>spi-xilinx-ps.c</i>
Arm global timer	<i>arm_global_timer.c</i>

Common Standalone Services

> Initialization and configuration

- The `X<Device>Ps_Config` structure is used by the driver to configure itself
 - Fields inside this structure are properties of `X<Device>Ps`, based on its hardware build
- `X<Device>Ps_LookupConfig (DeviceId)`
 - Looks up the device configuration based on the unique device ID. A table contains the configuration info for each device in the system
- `X<Device>Ps_CfgInitialize(InstancePtr, CfgPtr, EffectiveAddr)`
 - Initializes a specific `XSpiPs` instance such that the driver is ready to use
 - Uses a configuration structure provided by the caller. If running in a system with address translation, the parameter `EffectiveAddr` should be the virtual address

Common Standalone Services Example

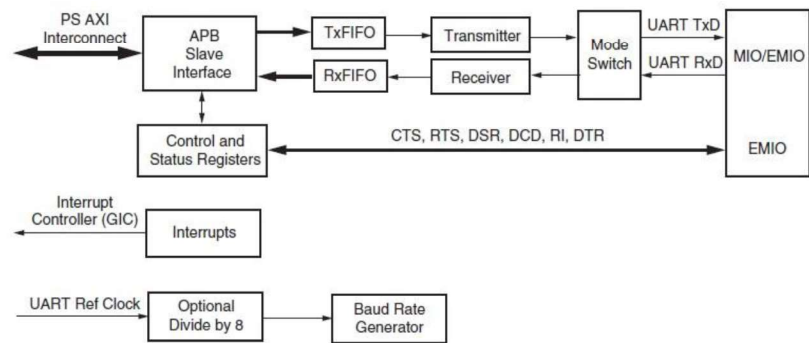
```
int UartPsSelfTestExample(u16 DeviceId)
{
    int Status;
    XUartPs_Config *Config;
    /*
     * Initialize the UART driver so that it's ready to use
     * Look up the configuration in the config table,
     * then initialize it.
     */
    Config = XUartPs_LookupConfig(DeviceId);
    if (NULL == Config) {
        return XST_FAILURE;
    }
    Status = XUartPs_CfgInitialize(&Uart_Ps, Config, Config->BaseAddress);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }
    /*
     * Perform a self-test to check hardware build.
     */
    Status = XUartPs_SelfTest(&Uart_Ps);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }
    return XST_SUCCESS;
}
```

Low-Speed Peripherals: UART

UART Controller

> The PS supports two UART devices in the IOP with the following key features

- Programmable baud rate generator
- 64-byte transmit and receive FIFOs
- Programmable protocol
 - 6, 7, or 8 data bits
 - 1, 1.5, or 2 stop bits
 - Odd, even, space, mark, and no parity
- Parity, framing and overrun error detection
- Interrupt generation
- Modem control signals: CTS, RTS, DSR, DTR, RI, and DCD are available only on the EMIO interface



>> Low-Speed Peripherals: UART - 1-2

© Copyright 2022 Xilinx

124029
XILINX

UART (1)

- > Two UARTs
- > Programmable baud rate generator
- > 64-byte receive and transmit FIFOs
- > 6, 7, or 8 data bits
- > 1, 1.5, or 2 stop bits
- > Odd, even, space, mark, or no parity
- > Parity, framing, and overrun error detection

>> Low-Speed Peripherals: UART - 1-3

© Copyright 2022 Xilinx

124029
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

UART (2)

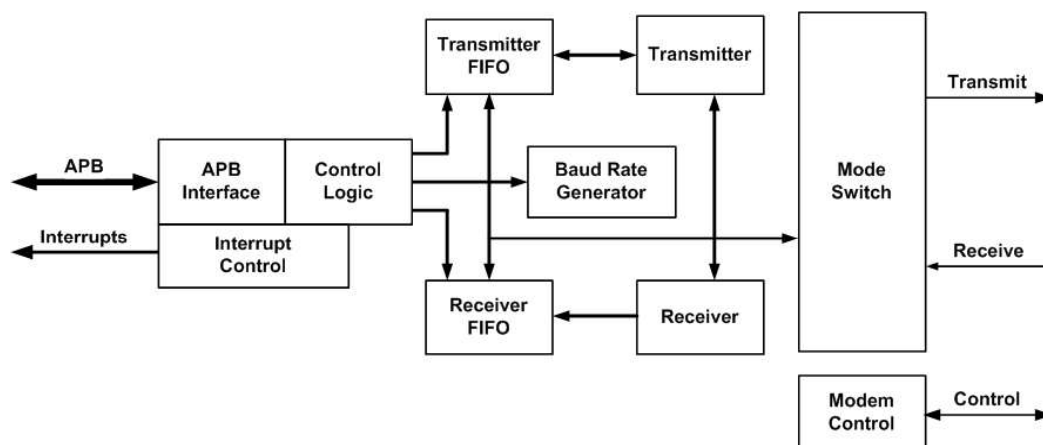
- > "Line break" generation and detection
- > Automatic echo, local loopback, and remote loopback channel modes
- > Interrupts generation
- > Support 8 Mb/s maximum baud rate with additional reference clock or up to 1.5 Mb/s with a 100-MHz peripheral bus clock
- > Modem control signals: CTS, RTS, DSR, DTR, RI, and DCD (through EMIO)
- > Simple UART: only two pins (TX and RX through MIO)

>> Low-Speed Peripherals: UART - 1-4

© Copyright 2022 Xilinx

124029
XILINX

UART Block Diagram



>> Low-Speed Peripherals: UART - 1-5

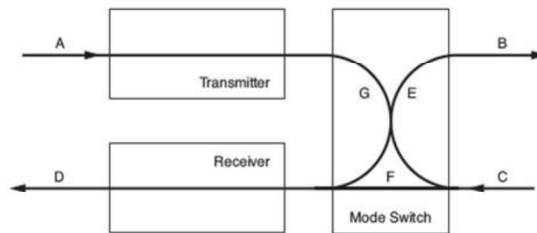
© Copyright 2022 Xilinx

124029
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

UART Operational Modes

- > Normal mode
- > Automatic echo mode
- > Local loopback
- > Remote loopback



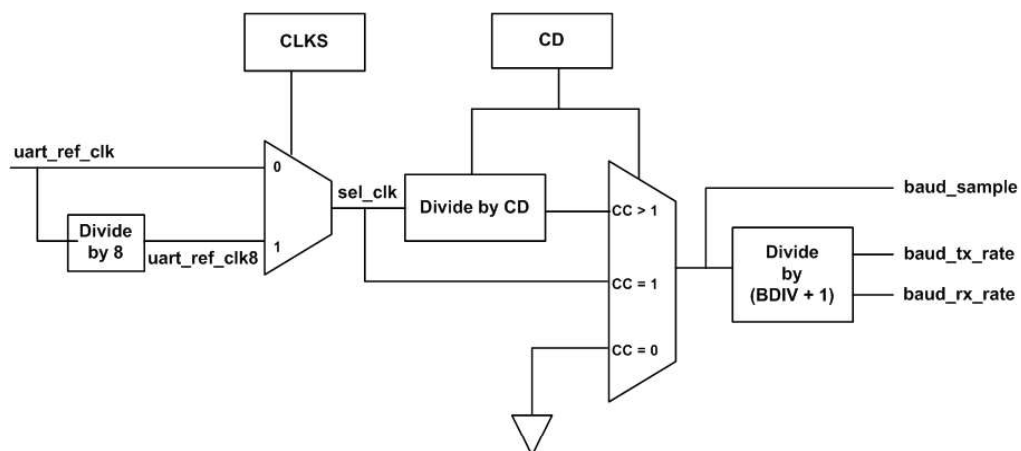
>> Low-Speed Peripherals: UART - 1-6

© Copyright 2022 Xilinx

124029
XILINX

UART Baud Rate Generator

- > Baud rate is set in the baud rate divider register



>> Low-Speed Peripherals: UART - 1-7

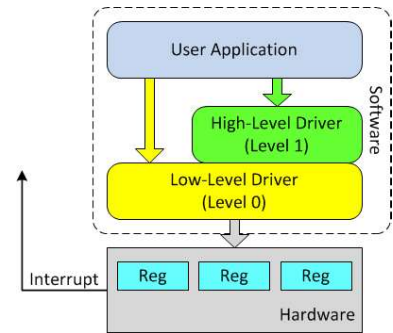
© Copyright 2022 Xilinx

124029
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Defining the Device Driver

- > A device driver
 - Is software
 - Typically sits just above the hardware
 - Translates user requests into bus transactions to the target
 - Manages register use
 - Timing
- > Comes in several *flavors*
 - Block: Moves blocks of data typically for disk drives
 - Character: Single symbol movements such as a serial terminal
 - Network: Transfers frames of data such as video other Ethernet
 - Pseudo-device: Does not drive hardware, but rather simulated devices (pseudo terminal)
- > Layered architecture provides seamless integration with the following
 - (Layer 2) RTOS adaptation layer: Provided by the OS; does not exist in a standalone system
 - (Layer 1) High-level device drivers that are full featured and portable across operating systems and processors
 - (Layer 0) Low-level drivers for simple use cases; direct hardware interface



Device Driver Description: UART

- > This driver supports the following features
 - Dynamic data format (baud rate, data bits, stop bits, parity)
 - Polled mode
 - Interrupt driven mode
 - Transmit and receive FIFOs (32 byte FIFO depth)
 - Access to the external modem control lines

Top-Level Standalone Driver APIs: UART

> Functions for setting UART parameters

- XUartPs_SetBaudRate()
- XUartPs_SetDataFormat()
- XUartPs_SetFifoThreshold()
- XUartPs_SetFlowDelay()
- XUartPs_SetHandler()
- XUartPs_SetInterruptMask()
- XUartPs_SetOperMode()
- XUartPs_SetOptions()
- XUartPs_SetRecvTimeout()

> Functions for reading UART parameters

- XUartPs_GetDataFormat()
- XUartPs_GetFifoThreshold()
- XUartPs_GetFlowDelay()
- XUartPs_GetInterruptMask()
- XUartPs_GetModemStatus()
- XUartPs_GetOperMode()
- XUartPs_GetOptions()
- XUartPs_GetRecvTimeout()

Using the UART APIs

> Use the UART APIs to generate the following program flow

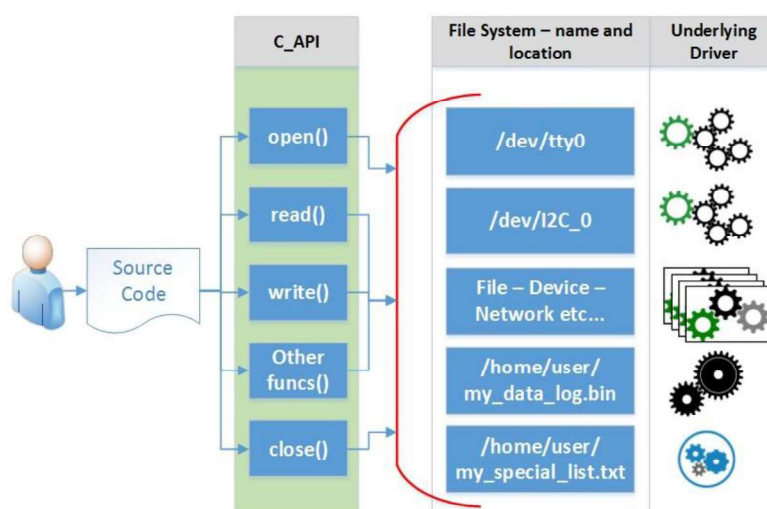
- Configure and initialize the UART data structure
- Set up the interrupt
- Send/receive the data

UART Sample Program

- > Leverage the provided example programs in the Vitis™ IDE for detailed examples using the aforementioned APIs
- > Example hyperlink located next to the UART device
- > Four programs are provided
 - Helloworld
 - Polled mode
 - Interrupt mode
 - Echo (character typed are echoed)

Linux Abstracted Driver Overview

- > Linux uses standard API to access 'files'
 - (In this case) Uses the POSIX C_API
- > Hardware devices presented as a 'file' on the file system
 - Standard way to access files
 - Complexity of underlying driver implementation is hidden
 - Should just work



Linux APIs: UART

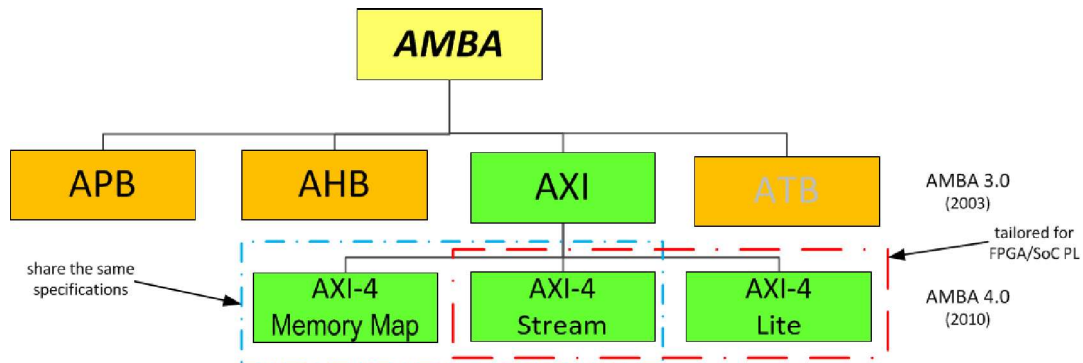
- > UART controller on the Zynq SoC is a 16550-compatible serial port
 - Drivers path: `/drivers/tty/serial/xilinx_uartps.c`
- > Linux device driver configuration is automatically determined by the device tree
 - No need to implicitly configure the driver to include it in the Linux kernel configuration
- > Uses standard Linux APIs to configure the UART port
 - Device present as `/dev/tty/S0`, `/dev/tty/S1` (can be modified as per requirements)
 - Use API functions: `open()`, `close()`, `read()`, `write()`, `ioctl()`
- > Refer to the Linux open-source documentation for more information



AXI: Introduction

2021.2

AXI is Part of AMBA: Advanced Microcontroller Bus Architecture



Interface	Features
Memory Map / Full	Traditional address/data burst (single address, multiple data)
Streaming	Data only, streaming
Lite	Traditional address/data—no burst (single address, single data only)

>> AXI: Introduction - 1-2

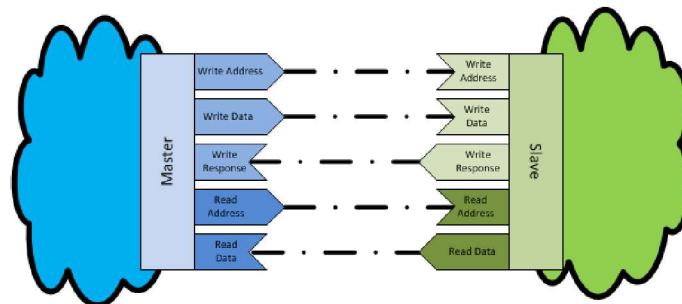
© Copyright 2022 Xilinx

111699



Arm AXI Functional Description

- > AXI is Advanced eXtensible Interface
 - Separate address, control, and data phases
 - Support for unaligned data transfers
 - Burst-based transactions
 - Issuing of multiple outstanding addresses
 - Easy to close timing with addition of register stages
- > Interconnect and protocol specification
 - Describes how two devices connect
- > The AXI specification describes an interface on a piece of IP
 - It does not specify how systems of IP will be connected



>> AXI: Introduction - 1-3

© Copyright 2022 Xilinx

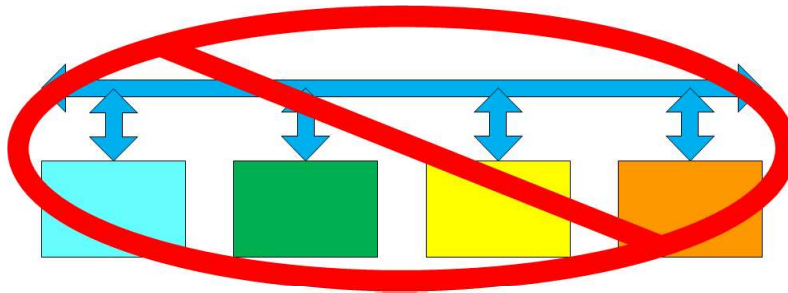
111699



DO NOT DISTRIBUTE PERSONAL USE ONLY

What Arm AXI Is Not

- > AXI is not a bus
 - Buses can be multi-tap/drop; AXI is only point-to-point
- > AXI is not an electrical specification
 - No discussion of voltages or other physical requirements

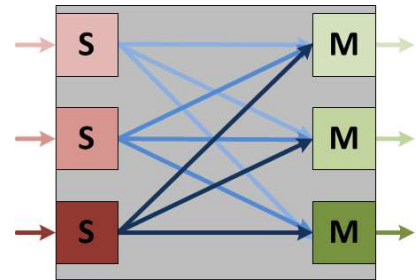


Goals of AXI

- > Higher performance
 - Enables system optimization for
 - Highest Fmax
 - Maximum throughput
 - Lower latency
- > Easier to use
 - Only one family of interfaces, regardless of the type of application or domain
- > Enable ecosystem
 - Partners are embracing the move to AXI
 - Open, widely adopted interface standard
 - Partner IP easily integrates into Xilinx tools
 - Common interface simplifies system construction yielding faster time to market

AXI System Architecture

- > AXI is a point-to-point protocol
 - One master port *talks* to only one slave port
- > This does not mean that one master device can talk to multiple slave devices, or vice versa
- > Special IP is used to augment AXI to support a full hierarchy
 - Crossbar: connects one or more similar AXI full masters to one or more slaves
 - Interconnect: similar to a crossbar, but provides for different clocks and resets



Documentation

- > *Xilinx AXI Reference Guide (UG1037)*
 - AXI for Xilinx system development
 - AXI support in Xilinx tools and IP with samples
 - AXI feature adoption in Xilinx FPGAs
 - Migrating to Xilinx AXI protocols
 - System optimization
 - Stream interoperability
- > Arm specification
 - AMBA AXI and ACE Protocol Specification (ARM IHI 0022D)
 - AMBA 4 AXI4-Stream Protocol Specification (ARM IHI 0051)
 - infocenter.arm.com/help/topic/com.arm.doc.set.amba
 - User registration required

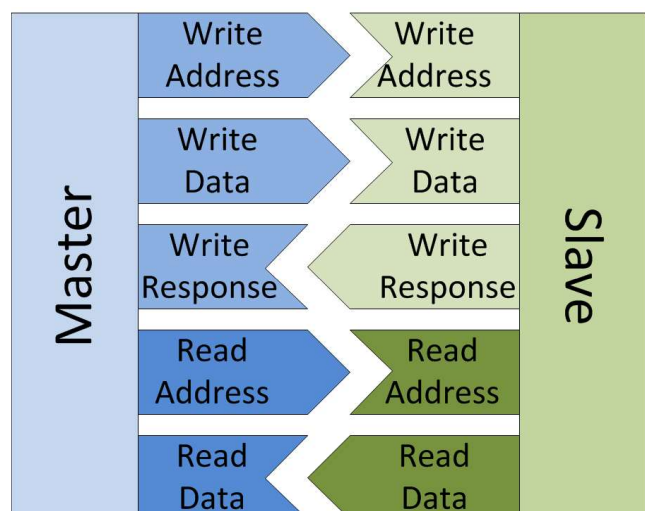
AXI: Variations

2021.2



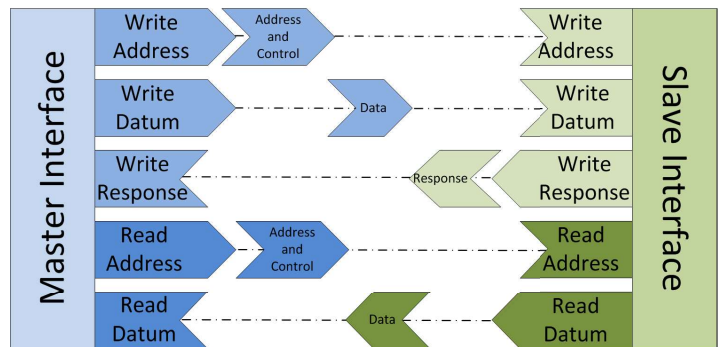
AXI (Full/Memory Mapped)

- > This is the base definition of AXI
 - Referred to as *full* or *memory mapped*
 - All other AXI types are variations from this definition
- > Contains all AXI features
 - Single address multiple data
 - Burst up to 256 data beats
 - Data width parameterizable
 - 32, 64, 128, 256, 512, 1024 bits
 - Xilinx limits the bus size to 256
 - Support for overlapped transactions



AXI-Lite

- > The AXI-Lite interface is a subset of the AXI interface
 - Intended for communication with control registers in components
- > Goals of AXI-Lite
 - Supports simple component interfaces
 - Requires fewer resources
 - Saves power and space, eases timing and routing
 - Easier to design and validate
- > Formal AXI-lite protocol enables
 - Components to be built using the same subset
 - Building of common *shim* to interface between AXI-Full and AXI-Lite



>> AXI: Variations - 1-3

© Copyright 2022 Xilinx

111750



AXI Signals: Full versus Lite

- > By limiting the transaction phase to a single datum rather than bursting, the number of supporting logic and signals is minimized

	AXI4	AXI4-Lite
Read Address	ARID	
	ARADDR	
	ARLEN	
	ARSIZE	
	ARBURST	
	ARLOCK	
	ARCACHE	ARCACHE
	ARPROT	ARPROT
	ARQOS	
	ARREGION	
Write Address	ARUSER	
	ARVALID	
	ARREADY	

	AXI4	AXI4-Lite
Read Data	RID	
	RDATA	RDATA
	RRESP	RRESP
	RLAST	
	RUSER	
	RVALID	
	RREADY	

	AXI4	AXI4-Lite
Write Address	AWID	
	AWADDR	
	AWLEN	
	AWSIZE	
	AWBURST	
	AWLOCK	
	AWCACHE	
	AWPROT	
	AWQOS	
	AWSIZE	
Write Data	AWREGION	
	AWLOCK	
	AWUSER	
	AWVALID	
	AWREADY	

	AXI4	AXI4-Lite
Write Data	WDATA	WDATA
	WSTRB	WSTRB
	WLAST	
	WUSER	
	WVALID	
	WREADY	
Write Resp.	BID	
	BRESP	BRESP
	BUSER	
	BVALID	
	BREADY	

>> AXI: Variations - 1-4

© Copyright 2022 Xilinx

111750



DO NOT DISTRIBUTE PERSONAL USE ONLY

AXI4 Master Transaction Implementation

- > AXI4 master transaction logic required for Xilinx attachments
 - Ready/valid for all channels
 - Transaction initiation logic
 - Logic to emit read and write channel address
 - Read and write data channel logic
 - Send or receive data
 - Response logic
 - Data burst counter
 - Last data detection
 - Error detection and handling logic

>> AXI: Variations - 1-5

© Copyright 2022 Xilinx

111750
 XILINX

AXI4 Slave Transaction Implementation

- > AXI4 slave transaction logic required for Xilinx attachments
 - Ready/valid for all channels
 - Latch target address based on burst type
 - Burst data phase counter (not needed for AXI4 Lite)
 - Transaction response

>> AXI: Variations - 1-6

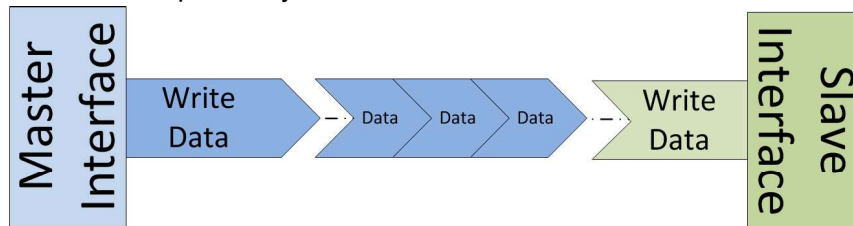
© Copyright 2022 Xilinx

111750
 XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

AXI Interface: Streaming

- > Data moves in a single direction, from a master to a slave
 - Simplex communication: master-to-slave
 - Dual direction would be dual simplex; individual channels, for example
- > Data only
 - No address channel
 - Minimum control
 - Unlimited burst length
 - Sideband signals optionally used to condition data
 - Contents of data stream interpreted by slave



>> AXI: Variations - 1-7

© Copyright 2022 Xilinx

111750
XILINX

AXI4 Stream Transaction Implementation

- > AXI4 stream transaction logic required for Xilinx attachments
 - Similar logic for stream master or slave
 - Ready/valid for all channels
 - Packet state machine: start and last data logic
 - Very much application dependent

>> AXI: Variations - 1-8

© Copyright 2022 Xilinx

111750
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

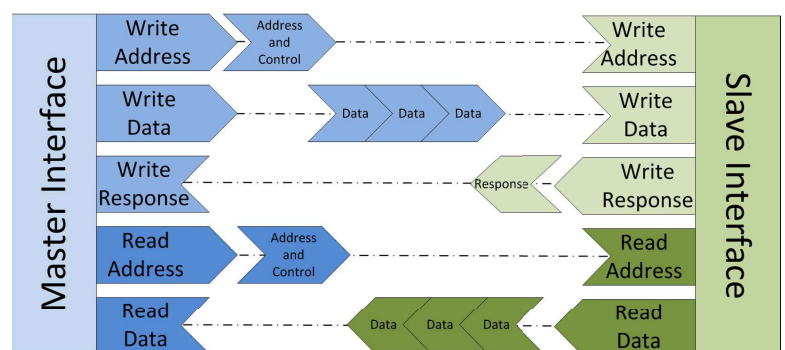
AXI: Transactions

2021.2



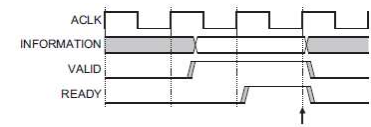
Memory-Map AXI Channels

- > Read address channel
- > Read data channel
- > Write address channel
- > Write data channel
- > Write response channel
 - Non-posted write model: there will always be a *write response*
- > Other signals are present to indicate when the information on the channel is valid, etc.

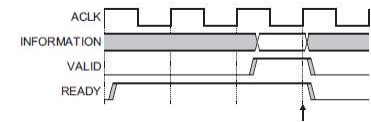


AXI Interface: Handshaking

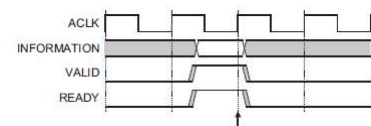
- > AXI uses a valid/ready handshake acknowledge
- > Each channel has its own valid/ready
 - Address (read/write)
 - Data (read/write)
 - Response (write only)
- > Flexible signaling functionality
 - Inserting wait states
 - Always ready
 - Same cycle acknowledge



Inserting Wait States



Always Ready



Same Cycle Acknowledge

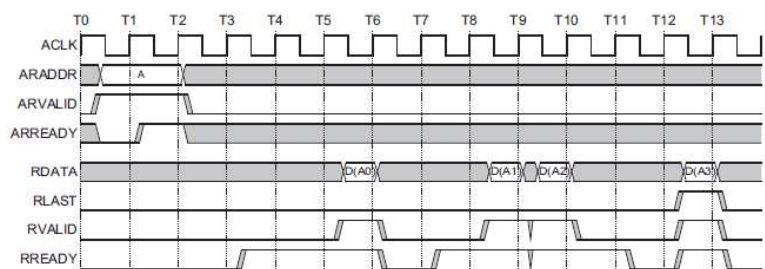
>> AXI: Transactions - 1-3

© Copyright 2022 Xilinx

111739
XILINX

AXI Interface: Memory-Map Read

- > Two channels
 - Address
 - Data
- > Up to 256 transfer data phase
- > Selectable data transfer size
- > See notes for signal detail of each channel



AXI – Burst Read

>> AXI: Transactions - 1-4

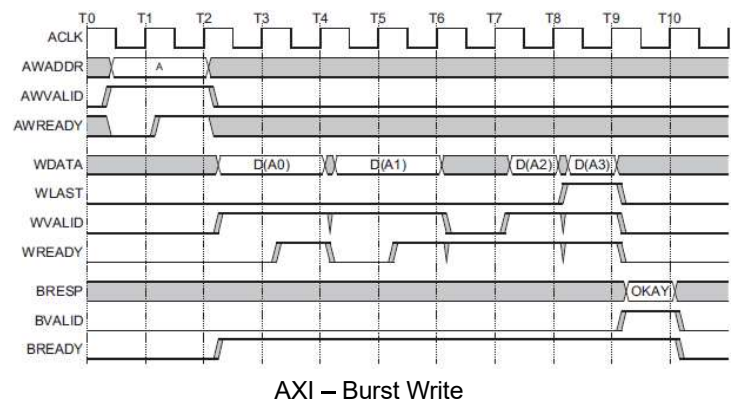
© Copyright 2022 Xilinx

111739
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

AXI Interface: Memory-Map Write

- > Three channels
 - Address
 - Data
 - Response
- > Up to 256 transfer data phase
- > Selectable data transfer size
- > See notes for signal detail of each channel



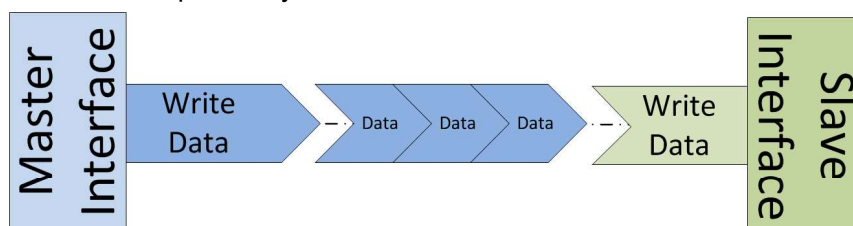
>> AXI: Transactions - 1-5

© Copyright 2022 Xilinx

111739
XILINX

AXI Interface: Streaming

- > Data moves in a single direction, from a master to a slave
 - Simplex communication: master-to-slave
 - Dual direction would be dual simplex; individual channels, for example
- > Data only
 - No address channel
 - Minimum control
 - Unlimited burst length
 - Sideband signals optionally used to condition data
 - Contents of data stream interpreted by slave



>> AXI: Transactions - 1-6

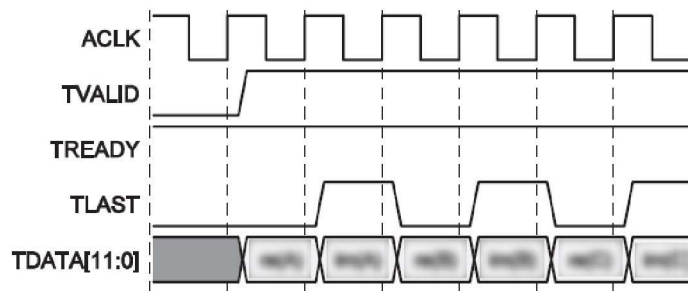
© Copyright 2022 Xilinx

111739
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

AXI Interface: Streaming Write

- > Only write data channel exists
 - Side-band signals are indicated when data is valid and when device is ready
- > Other side-band signals may be used for other needs
 - As required by the design



>> AXI: Transactions - 1-7

© Copyright 2022 Xilinx

111739



Zynq SoC PS-PL Interface

2021.2

DO NOT DISTRIBUTE PERSONAL USE ONLY

Interfacing to Programmable Logic

- > Most PS-PL interfacing is accomplished using some combination of the nine available AXI ports
 - Two general-purpose master ports
 - Two general-purpose slave ports
 - Four high-performance slave ports
 - One accelerator coherency port (ACP) slave port
- > AXI is a point-to-point protocol
 - A single master can connect to multiple slaves by using a switch
 - An understanding of PS interconnect (switch) operation is necessary for proper design use of the various ports
- > Other diagnostic, clock, and interrupt signals complete the PS-PL interface

>> Zynq SoC PS-PL Interface - 1-2

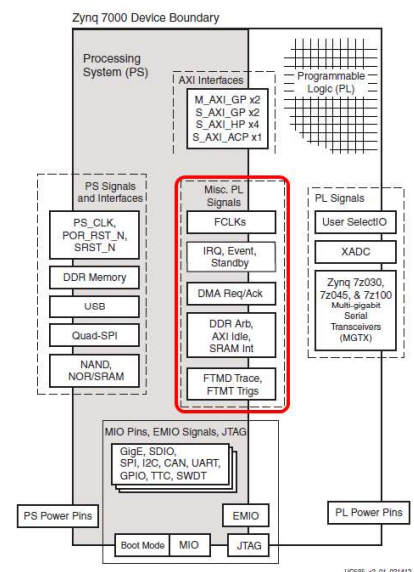
© Copyright 2022 Xilinx

133701



Miscellaneous PS-PL Signals

- > The PS-PL interface includes several miscellaneous signals outside of the primary AXI ports
- > These signals can be grouped into the following categories
 - Clock & Reset
 - Interrupts
 - Event
 - DMA
 - Trace / Debug
 - EMIO
 - Other



UG586_v2_01_021413

>> Zynq SoC PS-PL Interface - 1-3

© Copyright 2022 Xilinx

133701



DO NOT DISTRIBUTE PERSONAL USE ONLY

Clocks and Resets

> Clocks

- FCLK_CLK[3:0](O): Four clocks to be used as frequency sources in PL
 - Generated by PS clocking PLLs
 - Routed directly to PL clock buffers
 - No guaranteed phase relationship between the clocks
- FCLK_CLKTRIG_N[3:0](I): Signal to enable clock; glitch free and asynchronous to clock
 - Typically used for debug and development and power saving

> Resets

- FCLK_RESET_N[3:0](O): Four general reset signals from PS to PL
- Individually controlled by FPGA_RST_CTRL register in the SLCR

Interrupts

> Interrupt inputs from PL to PS

- 16 general-purpose interrupts to the GIC
- Two dedicated nIRQ; one to each CPU
- Two dedicated nFIQ; one to each CPU

> Interrupt outputs from PS to PL

- For use by a MicroBlaze processor or other specialized user designed hardware
- All of the IOP
- DMA controller (DMAC)
- SMC and QSPI memory controller

Understanding the PS-PL Interconnect Tradeoffs

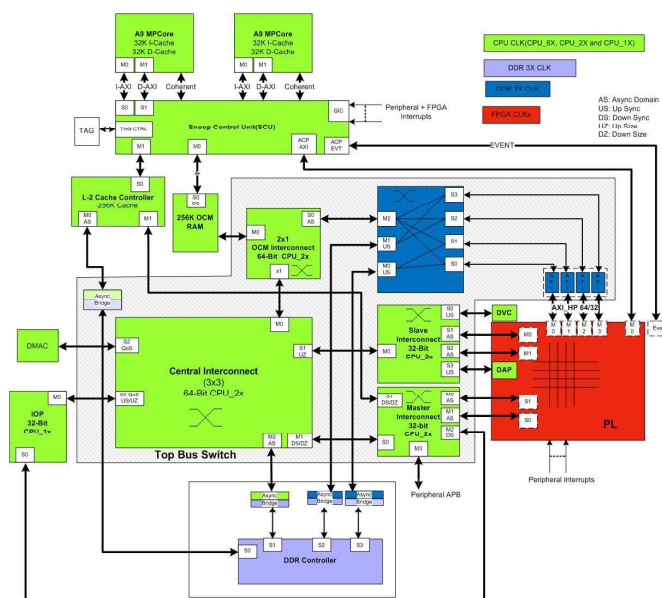
- > Understanding the PS interconnect facilitates a high-performance Zynq architecture design
- > The PS has six interconnect structures to facilitate data movement
- > Sometimes paths may need to make their way through multiple switches
- > Trade-offs exist between
 - PS peripheral access
 - Performance
 - PL AXI port access and loading
- > Pictures are on the following slides

>> Zynq SoC PS-PL Interface - 1-6

© Copyright 2022 Xilinx

133701
XILINX

PS Top-Level Interconnect Diagram



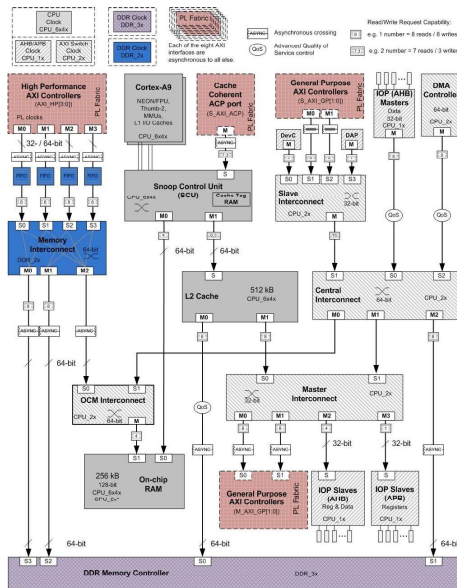
>> Zynq SoC PS-PL Interface - 1-7

© Copyright 2022 Xilinx

133701
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

PS Top-Level Interconnect Diagram



>> Zynq SoC PS-PL Interface - 1-8

© Copyright 2022 Xilinx

133701
XILINX

General-Purpose Master AXI Port Features

- > Each master is allocated a 1GB address space
 - Addresses start at 0x40000000 and 0x80000000, respectively
 - Only low-order 30 bits of AXI address are meaningful
- > PS masters can be
 - Cortex-A9 processors via the L2 cache controller
 - USB controller (2)
 - Ethernet controller (2)
 - SD/SDIO controllers (2)
 - DMAC
 - Debug access port
- > Mostly used for CPU and I/O peripheral block data movement **to** programmable logic

>> Zynq SoC PS-PL Interface - 1-9

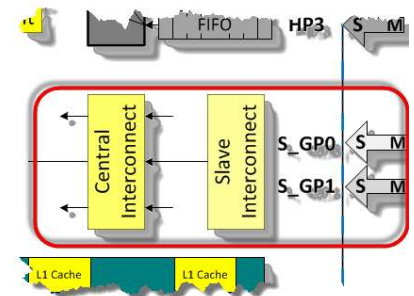
© Copyright 2022 Xilinx

133701
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

General-Purpose Slave Ports

- > Two identical 32-bit AXI slave ports
 - S_AXI_GP0
 - S_AXI_GP1
- > Provides initiator access from PL master to PS target
- > Attaches to master AXI port in PL
 - PL AXI interconnect
 - PL master
 - MicroBlaze processor
 - Your master built in programmable logic
 - Other third-party-based IP

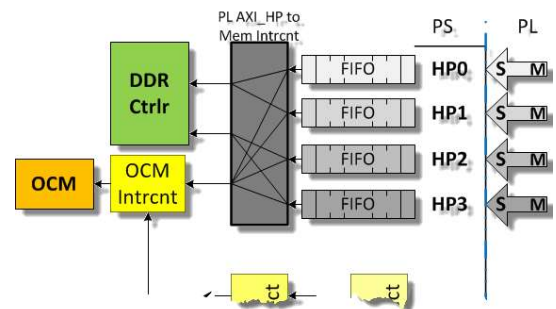


General-Purpose Slave AXI Port Features

- > Allows PL masters to access PS slaves
- > PS slaves can be
 - DDRx controller
 - On-chip memory (OCM)
 - IOP peripheral
 - Device configuration controller (DEVC)
 - Debug access port (DAP)
- > Mostly used for PL masters to access IOP, OCM RAM, and DDRx (simple access)

High-Performance Slave Ports

- > Four identical AXI slave ports
 - S_AXI_HP0
 - S_AXI_HP1
 - S_AXI_HP2
 - S_AXI_HP3
- > Provides initiator access from PL master to PS memory
- > Attaches to master AXI port in PL
 - DMA controller
 - MicroBlaze processor
 - Custom master built in programmable logic
 - Other third-party based IP



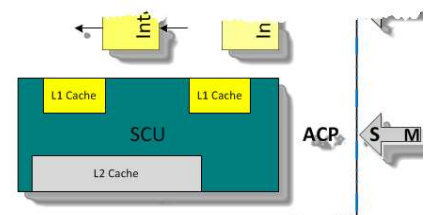
>> Zynq SoC PS-PL Interface - 1-12

© Copyright 2022 Xilinx

133701
XILINX

Accelerator Coherency Port (ACP) Slave Port

- > One accelerator coherence port (ACP)
 - S_AXI_ACP
- > Provides initiator access from the PL master to the PS target
 - Via L1 and L2 cache
- > Attaches to the master AXI port in PL
 - Typically a PL-based co-processor/accelerator
 - Could also be
 - A MicroBlaze processor
 - A custom master built in programmable logic
 - Another third-party-based IP



>> Zynq SoC PS-PL Interface - 1-13

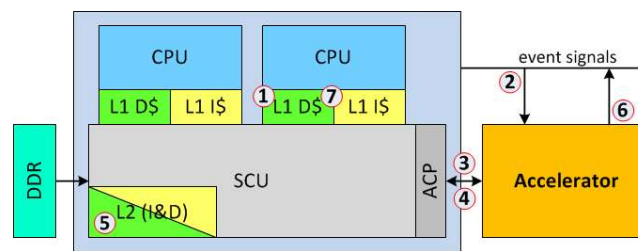
© Copyright 2022 Xilinx

133701
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY

Typical ACP Accelerator Interaction

- > Accelerator ties into Accelerator Coherency Port (ACP)
 - Accelerators gain access to CPU cache hierarchy
 - Compatible with standard un-cached peripherals and accelerators
 - Use of event signals optional
- > L2 "locked down" to limit high-throughput master flooding



>> Zynq SoC PS-PL Interface - 1-14

© Copyright 2022 Xilinx

133701
XILINX

What are the Differences Between Slave HPx, GPx, and ACP?

- > The four S_AXI_HP_x ports can only access the PS OCM and DMC
 - Purpose is low-latency PL to memory access
 - FIFOs built into interface allow for streaming
- > The two S_AXI_GP_x ports can access most PS peripherals, including memory
 - Purpose is programmable logic access to PS IOP and other PS slaves
 - Much slower access to OCM and DDR_x memory
- > Single S_AXI_ACP can access all PS memory and peripherals
 - Purpose is programmable logic co-processor memory interface
 - Performance advantage through cache hits
 - Lower latency than HP_x for cache misses

>> Zynq SoC PS-PL Interface - 1-15

© Copyright 2022 Xilinx

133701
XILINX

DO NOT DISTRIBUTE PERSONAL USE ONLY